

面向多源三维数据集自动适配的 SKILL.md 设计研究

调研结论与统一判断标准

你这张表里真正困难的点，不是“数据集多”，而是**数据载体、坐标系、标注粒度、资产形式、组织层级**同时不统一。公开数据里至少并存了几种完全不同的格式家族：

一类是 **JSON / CSV / SQLite / TSV** 这类元数据仓，比如 CO3D 的 `frame_annotations.jgz / sequence_annotations.jgz`、uCO3D 的 `metadata.sqlite` 与 `set_lists/*.sqlite`、Hypersim 的 `metadata_*.csv`；
一类是 **COLMAP 风格的多视图重建仓**，常见为 `cameras.txt|bin`、`images.txt|bin`、`points3D.txt|bin`，如 MVImgNet、MegaDepth、ScanNet++、DL3DV-10K benchmark；
一类是 **几何资产仓**，用 `ply / glb / usd / usdz / urdf / hdf5` 承载 mesh、point cloud、Gaussian、articulation 或 scene description，如 Replica、InteriorAgent、InteriorGS、YCB、Articraft-10K；
还有 **视频+文本** 的数据仓，例如 OpenVid-1M 的 `csv + mp4`；以及 **自动驾驶序列仓**，如 nuScenes 的 `samples/ sweeps/ maps/ v1.0-*.json`、KITTI 的 `png/bin/txt/xml`、Waymo 常见工程流中的 TFRecord/Proto 读取与预转换。¹

因此，适合 Agent 化的不是“为每个数据集写一堆硬编码 if-else”，而是把系统拆成两层：**容器解析层与语义归一层**。前者只负责读懂文件和目录；后者把“深度是沿射线还是沿相机 z 轴”“pose 是 OpenCV/COLMAP 还是 OpenGL/Nerfstudio”“标签是 LVIS、NYU40、Matterport40 还是 lane-line attributes”这类差异，统一映射到一个中间格式。这个判断并不是抽象的架构偏好，而是直接来自公开文档里对格式差异的明确描述，例如 ASE 深度以毫米存储且沿像素光线方向；Hypersim 同时存在 asset-units 与 meters 的换算；ScanNet++ 同时给出 COLMAP/OpenCV 和 Nerfstudio/OpenGL 两套相机表示；uCO3D、InteriorGS 则把三维重建从 mesh 扩展到了 point cloud / Gaussian 等更高阶载体。²

下面的调研，我会始终用同一个口径来描述每个数据集：**数据组织单位、核心文件类型、关键标注、坐标/单位注意事项、对自动转换的实际影响**。

室内场景与仿真资产数据集

这组数据集的共同特点是：很多都不是“直接给前馈训练样本”，而是给**场景资产、重建结果、仿真配置**。对 Agent 来说，关键不是下载后立刻训练，而是先判断它到底属于哪一类：**场景布局源、RGB-D 序列源、语义 mesh 源、仿真资产源、Gaussian 场景源**。Hypersim、ScanNet++、InteriorGS 分别代表了 HDF5 渲染仓、COLMAP+mesh+多传感器仓、3DGS 场景仓这三种典型模式。³

数据集	内容格式与组织方式	自动适配时的关键点	依据
3D-FRONT	以 室内场景布局 JSON 为核心，场景中的家具/物体实例通常引用 3D-FUTURE 类网格式资产；本质是 合成室内布局源 而不是现成 RGB-D 数据集。	应归类为 <code>scene_layout + referenced_mesh_assets</code> ； 适配器要先解析 JSON，再递归解析外部 mesh/材质引用，必要时再渲染 RGB / depth / mask。	⁴

数据集	内容格式与组织方式	自动适配时的关键点	依据
Aria Synthetic Environments	官方 data format 页面明确： <code>rgb</code> 为 JPEG ， <code>depth</code> 为 16-bit PNG ，像素值表示沿光线方向的深度、单位为 mm ；评测页还说明序列包含 RGB / depth / instance segmentation 。项目页说明它是过程化生成的大规模室内合成场景。	这是标准 <code>sequence + rgbd + instance mask</code> 数据；要特别处理“深度定义沿 ray，而非相机 z 轴”这一点； <code>pose / scene metadata</code> 建议统一入 <code>cameras</code> 与 <code>scene_meta</code> 。	5
ARKitScenes	官方 README 说明：包含 raw 和 processed data 、 camera pose 、 surface reconstruction ，并额外提供 stationary laser scanner 高分辨率 depth 与手工 3D oriented bounding boxes 。	适配器应把它识别成 <code>rgbd_sequence + pose + mesh + 3d_box</code> ；因为同时有移动设备低分辨率深度和静态激光高质量深度，建议输出成两个 profile： <code>mobile_rgbd</code> 与 <code>hr_depth_subset</code> 。	6
HM3D	HM3D 论文说明每个 scene 是 textured 3D mesh reconstruction ；Habitat 文档说明下载时可获得 Habitat-ready 资产，并可附带 HM3D-Semantics。Matterport 页面还说明原始网格可含 OBJ / GLB + texture maps + MTL 。	这类数据更适合作为 <code>habitat_scene_asset</code> 或 <code>mesh_scene</code> ，而不是直接的前馈样本；若需要训练数据，应通过 viewer/simulator 再渲染 RGB/Depth/semantic。	7
Hypersim	每个 scene 一个 ZIP； <code>metadata_cameras.csv</code> 、 <code>metadata_scene.csv</code> 、相机位姿 <code>camera_keyframe_*.hdf5</code> ，图像和几何以 HDF5 保存；还含 NYU40 语义、instance id、9-DOF bbox 。坐标既有 asset coordinates，也有 <code>meters_per_asset_unit</code> 。	这是最典型的 <code>rendered_scene_archive</code> ；要优先做 HDF5 reader、asset-unit→meter 归一、rotation matrix 解析 ；若中间层不支持“单位换算”和“pose convention”，后面全部都会错。	8
Matterport3D	官方 repo 写明包含 color/depth images、camera poses、textured 3D meshes、floor plans、region annotations、object instance semantic annotations ，并指向 data organization 文档；下载需签署 ToU。	适配时可同时支持 <code>rgbd_sequence</code> 和 <code>mesh_scene</code> 两种导出；但要 把授权门槛纳入 Agent 检查逻辑 ，未满足 ToU 时不能假设可自动下载。	9
Replica	官方 README 与论文都说明：每个场景有 dense mesh、HDR texture、glass/mirror info、planar segmentation、semantic class/instance segmentation ；Habitat 导出目录中可见 <code>mesh_semantic.ply</code> 、 <code>info_semantic.json</code> 等。	应识别为 <code>semantic_mesh_scene</code> ；如果目标是前馈训练，需要二次渲染；如果目标是具身仿真，则优先保留 Habitat 导出格式。	10

数据集	内容格式与组织方式	自动适配时的关键点	依据
ReplicaCAD	官方页面给出更细的仿真资产结构：有 3D object assets 、 stage assets 、 URDF articulated furniture 、 SceneDataset config 、 .navmesh 。	这是 <code>interactive_sim_scene</code> ，不是普通 mesh 数据集；中间格式必须支持 rigid object / articulated object / navmesh / receptacle metadata 。	11
ScanNet v2	官方 README 明确每个 scan 目录包含 <code>.sens</code> 、 <code>*_vh_clean*.ply</code> 、 <code>*.segs.json</code> 、 <code>*.aggregation.json</code> 、 <code>*_2d-label*.zip</code> 、 <code>*_2d-instance*.zip</code> ；并解释 <code>.sens</code> 与 <code>aggregation.json</code> 等格式。	适配器至少支持 <code>.sens</code> 解包、 <code>mesh + segIndices + segGroups</code> 合并、2D/3D 标注对齐。	12
ScanNet++	官方文档给出最完整的数据树： <code>scans/pc_aligned.ply</code> 、 <code>mesh_aligned_0.05_semantic.ply</code> 、 <code>segments.json</code> 、 <code>segments_anno.json</code> ， <code>dslr/colmap/{cameras.txt, images.txt, points3D.txt}</code> ， <code>nerfstudio/transforms.json</code> ， <code>iphone/rgb.mkv</code> 、 <code>depth.bin</code> 、 <code>pose_intrinsic_imu.json</code> 等。	对 Agent 很关键的一点：同一 scene 同时暴露 COLMAP/OpenCV 与 Nerfstudio/OpenGL 两套相机格式，非常适合作为“坐标系变换回归测试集”；scene graph 提取更适合作为导出目标而非原始模式。	13
InteriorAgent	Hugging Face 卡片把它定义为 高质量 3D USD 资产集合 ，带 modular materials 、 scene description files 、 physics-ready geometry ，面向 Isaac Sim。	直接按 <code>usd_scene_asset</code> 处理；不要误当作 RGB-D 数据集。最合适的导出目标是 USD/Omniverse/IsaacSim profile。	14
InteriorGS	官方卡片给出目录：每 scene 有 <code>3dgs_compressed.ply</code> 、 <code>labels.json</code> 、 <code>occupancy.png</code> 、 <code>occupancy.json</code> 、 <code>structure.json</code> ；并说明 PLY 存的是 Gaussian 参数，labels 是 object-level bbox/instance label。	这是 <code>gaussian_scene + semantics + occupancy + floorplan</code> 的标准范例；中间层必须把 Gaussian scene 当作一级公民，而不是强制先转 mesh。	15
SAGE-10k	数据卡片说明根目录是 <code>scenes/scene_id/</code> ，内含 <code>objects/</code> 、 <code>materials/</code> 、 <code>preview/</code> 、 <code>layout_id.json</code> ；项目强调其是 simulation-ready interactive indoor scene dataset 。	应归为 <code>generated_scene_asset</code> ；重点读取 <code>layout JSON + object assets + materials</code> ，再根据目标导出为 GLB / USD / Isaac Sim scene。	16
Tabletop_Scenes	Hugging Face 明确： <code>scene_gallery/</code> 内是 <code>.glb</code> 的桌面 3D 场景， <code>manipulation_demo/</code> 是 Isaac Sim 操作演示代码与资产。	适配非常直接： <code>glb_scene_asset</code> ；必要时补一个 <code>tabletop_task_meta</code> 导出层。	17

数据集	内容格式与组织方式	自动适配时的关键点	依据
Maya	你表里没有官方链接，而且“Maya”不是社区唯一对应的公开 benchmark 名称。	建议不要先写死专用 adapter，而是先按 自定义 mesh / DCC 场景源 处理，等待补充正式链接后再产品化。	基于你提供的表格信息作出的实现判断

物体级、多视图与生成资产数据集

这一类的典型分歧在于：有的是**每个对象一个多视图序列**，有的是**每个场景一个 COLMAP 重建**，有的是**每个资产一个可执行 URDF / GLB / Gaussian**。如果不先把它们拆成 `object_sequence`、`scene_sequence`、`asset_object` 三大类，后续“统一格式”一定会混乱。CO3D、uCO3D、MVIImgNet、Objaverse-XL、Articraft-10K，实际上分别代表了五种不同的对象级数据哲学。¹⁸

数据集	内容格式与组织方式	自动适配时的关键点	依据
BlendMVS	<code>rmvd</code> 文档说明其下载后就是 scene folders ，并被统一读取为多视图深度样本； <code>mvd</code> 数据格式要求输出 <code>images / poses / intrinsics / keyview_idx / depth</code> 。	对你的系统而言，最合理的是直接把它归入 <code>multiview_scene_mvs</code> ；不要过度抽象，按 keyview-style 导出最实用。	¹⁹
CO3D v1/v2	官方 README 清楚给出：每个 category/sequence 下有 <code>images/ depths/ masks/ depth_masks/ pointcloud.ply</code> ；全类别元数据是 <code>frame_annotations.jgz</code> 和 <code>sequence_annotations.jgz</code> ；set lists 和 eval batches 也是 JSON。原始 dataclass 里明确有 <code>image / depth / mask / viewpoint / point_cloud</code> 。	这是 <code>object_multiview_sequence</code> 的黄金标准；建议把 <code>FrameAnnotation</code> 原样映射到中间层，而不是自创字段名。	²⁰

数据集	内容格式与组织方式	自动适配时的关键点	依据
DL3DV-10K	项目页和论文说明：它提供 4K videos / RGB images / calibrated camera poses / human scene labels ；GitHub 数据页说明 benchmark 提供 Nerfstudio 与 3DGS 格式 。	这类数据天然适合 <code>scene_multiview_reconstruction</code> ；你的 exporter 最好原生支持 <code>nerfstudio/transforms.json</code> 与 <code>3dgs</code> 两个出口。	21
MegaDepth	官方页说明深度图来自 COLMAP SfM/MVS ；同时提供 COLMAP 与 Bundler 格式 的 SfM models，含 sparse points 和 intrinsics/extrinsics。	应按 <code>internet_photo_multiview_scene</code> 处理；COLMAP reader 是核心依赖。	22
MVImgNet	官方 GitHub 给出目录： <code>ROOT/class_label/instance_id/images</code> 与 <code>sparse/0/{cameras.bin, images.bin, points3D.bin}</code> ；下载工具还支持只拉 <code>MVImgNet_mask</code> 等子集。	适合 <code>object_multiview_colmap</code> profile；类目与实例分层天然适配 <code>category / object_id / views</code> 三段式中间键。	23
MVImgNet 2.0	官网说明新增更多 360° 拍摄、改进 masks、改进 SfM pose，并增加高质量 dense point clouds；仍覆盖 object masks / camera parameters / point clouds。	与 MVImgNet 可共用一个 adapter，但要在 registry 中区分 <code>version</code> 与 <code>dense_pointcloud_available=true</code> 。	24
Objaverse-XL	官方站点把它定义为 10M+ 3D objects ；Objaverse API 文档说明按 UID 获取 annotations / metadata；1.0 文档还展示了对对象 annotation 的典型字段（name、tags、thumbnails、uri 等），并说明 1.0 对象可经 XL API 下载。	这类数据不是 sequence，而是 <code>asset_bank</code> ；中间格式应强调 <code>asset_id / source / raw_mesh_format / metadata / text tags</code> 。	25
OpenVid-1M	官方 repo 给出目录： <code>data/train/OpenVid-1M.csv</code> 、 <code>OpenVidHD.csv</code> 与 <code>video/*.mp4</code> ；论文说明它是 1M+ text-video pairs 。	这是 <code>video_text_dataset</code> ；如果你的目标格式面向三维/具身任务，只应把它作为 文本-视频预训练源 ，不应硬塞进 scene mesh schema。	26
StaticThings3D	<code>robustmvd</code> 文档说明需下载后转换目录结构；其统一 <code>mvd</code> 输出就是 <code>images / poses / intrinsics / depth</code> 。	最适合做 转换器回归测试集 ，因为它结构简单、字段清晰。	19

数据集	内容格式与组织方式	自动适配时的关键点	依据
uCO3D	官方 README 给出完整文件树： metadata.sqlite、set_lists/ *.sqlite，每 sequence 下有 rgb_video.mp4、 mask_video.mkv、 depth_maps.h5、 point_cloud.ply、 segmented_point_cloud.ply、 sparse_point_cloud.ply、 gaussian_splats；并说明带长短 caption、LVIS taxonomy 类别。	这是最适合你系统做“对象级一站式 schema”的数据：视频、mask、 depth、pose、point cloud、 Gaussian、text 全都有。优先级应该 非常高。	27
YCB Benchmark	Habitat 版目录为 configs/ *.object_config.json、meshes/ *.glb(.orig)、collision_meshes/ *.glb、 ycb.scene_dataset_config.json； 原始 YCB 官方页说明含 mesh models 与高分辨率 RGB-D scans。	适配时最好拆成两个 profile： raw_ycb_rgbd_object 与 habitat_ready_object_asset。	28
Articraft-10K	Hugging Face 数据卡说明它是 10k articulated 3D objects in URDF format ；论文页说明总量超过 10K、覆盖 245 categories ，由 Articraft agent 生成。	这是 articulated_asset 的直接样 本；中间层必须原生支持 joint/link/ limit/material/collision。	29

自动驾驶与遥感地图数据集

这一组数据的差异，不在于“有没有图像”，而在于**时间同步、坐标系层级、地图表示**。nuScenes、KITTI、KITTI-360、Waymo、WayveScenes101 属于连续时序的车载采集；OpenSatMap、seed-map、3D-GloBFP 则是遥感/地图数据，核心对象从“frame”变成了 **polyline、polygon、height raster**。如果你的中间格式不把“时序序列”和“地理矢量”分成两类，后面很难稳定扩展。 30

数据集	内容格式与组织方式	自动适配时的关键点	依据
BDD100K	官方论文说明覆盖 10 个驾驶任务 ；配套准备文档说明任务按 annotation bundle 下载，例如 Detection 2020 Labels、Instance Segmentation、Semantic Segmentation；100K image 版本来自视频第 10 秒的关键帧。	应按 driving_image_dataset 与 driving_video_dataset 双 profile 处理；同一个 dataset name 下要允许多套 task-specific exporter。	31

数据集	内容格式与组织方式	自动适配时的关键点	依据
nuScenes	官方 devkit 结构为 samples/、sweeps/、 maps/、v1.0-* JSON tables; 官方 schema 明确 包含 sample_data、 ego_pose、sensor、 calibrated_sensor、 sample_annotation; 官方 介绍强调 6 cameras + 1 lidar + 5 radar + GPS/IMU。	中间层应直接保留 scene / sample / sample_data / ego_pose / calibrated_sensor 这套关系模型，不要强行扁平化。	32
KITTI	官方 raw data 页给出：左右 灰度/彩色图像为 PNG ， Velodyne 为 binary float matrix ，GPS/IMU 为 text ， calibration 为 text ， tracklets 为 XML 。	适合 driving_stereo_lidar_sequence ；最关键的是保留 calib + oxts + velodyne + image_02/03 原始层次。	33
KITTI-360	官方说明传感器包括一对 perspective camera、一对 fisheye camera、Velodyne 和 SICK；文档还给出了 2D semantics 的路径与 8-bit PNG semantic / instance 约定 。	这是 driving_multicamera_mapping_sequence ；比 KITTI 多出 fisheye 与更强的 2D/3D 语义，需要 单独 adapter，而不是“兼容 KITTI 即可”。	34
Wayve	若你表中的“wayve”指公 开可用的 WayveScenes101 ，官方页 面说明它含 101 scenes、 five time-synchronised vehicle-mounted cameras 与 camera poses ，服务于 novel view synthesis / scene reconstruction。	建议在 registry 中把它注册为 wayve_scenes101 ，不要只写 wayve ；因为 “Wayve”作为公司名太宽泛。	35
Waymo Open Dataset	官方 about 页说明有 sensor calibrations、vehicle poses、12.6M 3D lidar boxes、11.8M 2D camera boxes、2D video panoptic segmentation ；社区训练管 线通常先把原始数据从 TfRecord 读取并预转换为 KITTI-like 格式。	工程上应该把 Waymo 视为 proto/tfrecord- backed sequence dataset ；Agent 第一阶段不 必自己实现全量 Proto 解析，可以先调用成熟 reader，再统一映射。	36

数据集	内容格式与组织方式	自动适配时的关键点	依据
OpenSatMap	官方页说明提供高分辨率卫星图，给出 vectorized polylines 表示 line instance，并给每条线 8 个 attributes；对齐 nuScenes 与 Argoverse 2。	这不是普通 segmentation mask 数据，而是 <code>satellite_image + vector_polyline + attributes</code> ；中间层要支持 <code>polyline</code> 与 <code>per-instance attributes</code> 。	37
seed-map	官方仓库说明原始包是 <code>image-label pairs</code> ；同时提供 COCO form 与 ADE20K form ；构建流程是从 NGII shapefile → JSON → coordinate list → labels → image alignment 。	这是非常适合做 Agent 演示的数据，因为它公开地展示了从 矢量地图到模型训练格式 的转换链路。	38
3D-GloBFP	官方 R 包 README 说明数据覆盖全球城市区域，托管为 shapefile format ；下载与查询输出可得 3D building footprint sf polygons 、building presence raster、height raster（米）。论文/Zenodo 说明它是 building-footprint-level 的全球建筑高度数据。	归类为 <code>geospatial_polygon_height_dataset</code> ；中间层应支持 <code>polygon + height + rasterized views</code> 双表示。	39

统一目标格式设计

基于上面的调研，我不建议你只定义一个“万能样本格式”。更稳妥的做法，是定义一个**统一中间格式**，再在其上派生几个**任务导出 profile**。原因很直接：`uCO3D` 的一个逻辑单元是“对象视频序列”，`Matterport3D` 的逻辑单元是“建筑场景”，`Objaverse-XL` 的逻辑单元是“单个 3D 资产”，`OpenSatMap` 的逻辑单元则是“航拍底图 + 矢量折线实例”。这些实体本体都不同，如果一开始就强行扁平，会把系统变脆。这个结论可以从 CO3D/uCO3D 的 sequence 元数据、ScanNet++ 的 scene 结构、Objaverse 的 asset API、OpenSatMap 的 polyline attributes 直接看出来。40

我建议你的统一中间层采用下面这套结构，名字可以叫 `UnifiedEmbodiedRecord`：

```
{
  "dataset": "string",
  "version": "string",
  "entity_type": "scene | sequence | object | map_tile | articulated_asset",
  "split": "train | val | test | custom",
  "ids": {
    "scene_id": "string|null",
    "sequence_id": "string|null",
    "object_id": "string|null",
    "frame_id": "string|null",
    "sample_id": "string"
  }
}
```



```

},
"time": {
  "timestamp": "float|null",
  "frame_index": "int|null",
  "fps": "float|null"
},
"storage": {
  "root": "string",
  "relative_paths": {}
},
"modalities": {
  "rgb": [],
  "depth": [],
  "mask_2d": [],
  "semseg_2d": [],
  "instance_2d": [],
  "pointcloud": [],
  "mesh": [],
  "gaussians": [],
  "video": [],
  "text": [],
  "map_vectors": [],
  "occupancy": [],
  "navmesh": []
},
"camera": {
  "intrinsics": [],
  "extrinsics": [],
  "pose_convention": "opencv|colmap|opengl|arkit|custom",
  "depth_convention": "z_axis|ray_distance",
  "unit": "m|mm|asset_unit"
},
"annotation": {
  "boxes_2d": [],
  "boxes_3d": [],
  "instances_3d": [],
  "semantic_labels": [],
  "articulation": [],
  "captions": [],
  "attributes": [],
  "scene_graph": []
},
"taxonomy": {
  "source_taxonomy": "string",
  "canonical_taxonomy": "string",
  "mapping_file": "string|null"
},
"geo": {
  "crs": "string|null",
  "tile_id": "string|null",
  "extent": null
}

```

```

},
"quality": {
  "is_synthetic": true,
  "has_metric_scale": true,
  "is_sim_ready": false,
  "is_forward_feed_ready": false
}
}

```

在这个中间层之上，再定义五个导出 profile 就够了：

导出 profile	适合的数据集	典型输出
<code>vision_sequence</code>	BDD100K、KITTI、KITTI-360、nuScenes、Waymo、WayveScenes101	帧索引 + 相机/激光器 + 2D/3D 标注
<code>multiview_recon</code>	BlendMVS、CO3D、MVLingNet、MegaDepth、DL3DV-10K、StaticThings3D	views + intrinsics + extrinsics + keyview depth
<code>indoor_scene</code>	ARKitScenes、Hypersim、Matterport3D、ScanNet、ScanNet++、Replica	scene + mesh/point cloud + pose + semantic/instance
<code>asset_bank</code>	Objaverse-XL、YCB、InteriorAgent、SAGE-10k、Articraft-10K	asset metadata + geometry + material + articulation
<code>geo_map</code>	OpenSatMap、seed-map、3D-GloBFP	image/tile + polyline/polygon + attributes + raster views

这五种 profile 解决了你后续所有“要不要构建前馈数据集”的问题：

前馈数据集不是输入层定义，而是导出层定义。

也就是说，像 3D-FRONT、ReplicaCAD、InteriorAgent、SAGE-10k 这类原本更像“场景资产库”的数据，完全可以通过导出 profile 再生成前馈训练样本；而像 BDD100K、KITTI、BlendMVS 则天然就是前馈友好的。

SKILL.md 方案与实施流程

如果你的目标是“利用 Agent 技术，搭一个 SKILL.md，自动把不同数据集转换成目标格式”，那 SKILL 不能只是写“读取数据集并转换”。它必须明确三件事：

第一，Agent 的工作对象不是样本，而是“数据集描述 + 解析器 + 目标 profile”。

第二，Agent 必须先做 schema 识别，再做转换。

第三，所有 dataset-specific 逻辑都必须沉淀进 registry，而不是埋在 prompt 里。

下面这份 `SKILL.md` 我建议直接作为可执行规范起点：

```
# SKILL: dataset_adaptation
```

```
## Goal
```

```
将异构公开数据集自动识别、解析、归一化，并导出为指定目标数据集格式。
```

```
## Inputs
```

```
- dataset_name
```

- dataset_root_or_url
- target_profile
- target_taxonomy
- export_root

Supported entity families

- indoor_scene
- multiview_recon
- driving_sequence
- asset_bank
- geo_map
- video_text

Required capabilities

1. Inspect dataset card / README / docs / local tree
2. Detect file carriers: json, csv, sqlite, hdf5, txt/bin(colmap), sens, tfrecord, ply, glb, usd/usdz, urdf, mp4/mkv, png/jpg
3. Resolve coordinate system and metric units
4. Map source taxonomy to canonical taxonomy
5. Export canonical manifest + materialized target profile
6. Run validation and produce QA report

Decision policy

- 优先使用 dataset registry 中的显式规则
- 若 registry 无命中, 则执行 format sniffing
- 若 sniffing 仍不确定, 则输出“半自动模式”并生成待确认字段
- 不得在未确定坐标系/单位时导出 3D 标注

Output artifacts

- canonical_manifest.jsonl
- schema_report.json
- taxonomy_map.json
- conversion_log.json
- qa_report.json
- exported_dataset/

Workflow

1. Identify dataset family
2. Load dataset descriptor from registry
3. Parse source containers
4. Normalize paths / ids / split / timestamps
5. Normalize cameras / depth / units / poses
6. Normalize semantics / instances / 3D structures
7. Export profile-specific dataset
8. Run validators
9. Emit report and unresolved warnings

Failure rules

- 缺关键元数据时停止导出 3D supervision
- 仅当许可允许且文件可访问时执行自动下载
- 对门控数据集只生成下载说明与待处理清单, 不伪造内容

仅有 `SKILL.md` 还不够，还要配一套 **Registry + Adapter + Validator** 的代码骨架。我建议目录这样设计：

```
project/
├── SKILL.md
├── dataset_registry/
│   ├── 3d_front.yaml
│   ├── ase.yaml
│   ├── arkitscenes.yaml
│   ├── bdd100k.yaml
│   ├── nusenes.yaml
│   ├── scannetpp.yaml
│   ├── uco3d.yaml
│   └── ...
├── parsers/
│   ├── json_parser.py
│   ├── csv_parser.py
│   ├── sqlite_parser.py
│   ├── hdf5_parser.py
│   ├── colmap_parser.py
│   ├── sens_parser.py
│   ├── tfrecord_parser.py
│   ├── mesh_parser.py
│   ├── usd_parser.py
│   └── urdf_parser.py
├── adapters/
│   ├── indoor_scene_adapter.py
│   ├── multiview_adapter.py
│   ├── driving_adapter.py
│   ├── asset_adapter.py
│   └── geo_adapter.py
├── normalizers/
│   ├── pose_normalizer.py
│   ├── depth_normalizer.py
│   ├── taxonomy_mapper.py
│   ├── unit_normalizer.py
│   └── path_resolver.py
├── exporters/
│   ├── export_vision_sequence.py
│   ├── export_multiview_recon.py
│   ├── export_indoor_scene.py
│   ├── export_asset_bank.py
│   └── export_geo_map.py
└── validators/
    ├── validate_pose.py
    ├── validate_depth.py
    ├── validate_taxonomy.py
    ├── validate_geometry.py
    └── validate_split_integrity.py
```

真正落地时，实施顺序不要按“数据集名称”排，而要按**格式共性**排。最有效的路线是下面这样：

阶段	目标	先做哪些数据集	交付件
基础容器层	打通 json/csv/sqlite/ txt/bin(colmap)/ply/ png/jpg/mp4	CO3D、MVLmgNet、DL3DV-10K、 OpenVid-1M、OpenSatMap、seed- map	registry v1、parser v1、 canonical manifest v1
多视图与相机层	统一 intrinsics/ extrinsics/depth/unit	BlendMVS、MegaDepth、 StaticThings3D、ARKitScenes、 Hypersim、ScanNet++	pose/depth normalizer、camera validator
车载序列层	统一时序 sample、车辆 pose、传感器标定	BDD100K、nuScenes、KITTI、 KITTI-360、WayveScenes101	driving exporter、 sample-index schema
三维资产层	统一 mesh / glb / usd / urdf / gaussian	Objaverse-XL、YCB、 InteriorAgent、InteriorGS、 SAGE-10k、Articraft-10K	asset schema、 articulation schema、 gaussian schema
高优先级增强层	做你表中“中高/高优先级”的完整闭环	BDD100K、BlendMVS、nuScenes、 ARKitScenes、ASE、KITTI、 KITTI-360、Waymo、Wayve	可复用 E2E pipeline
特殊与地理层	补 geospatial / scene- graph / gated datasets	3D-GloBFP、OpenSatMap、seed- map、HM3D、Matterport3D、 ScanNet++ scene graph	geo exporter、scene- graph exporter

从投入产出比看，我建议你的**首批必做名单**是：

1. **BDD100K、nuScenes、KITTI、KITTI-360**：因为它们最能验证“前馈序列 + 2D/3D 标注 + pose/calib”的通用性。⁴¹
2. **BlendMVS、CO3D、MVLmgNet、DL3DV-10K**：因为它们最能验证“多视图重建 profile”的稳固性。⁴²
3. **ARKitScenes、ASE、Hypersim、ScanNet++**：因为这组最能暴露 RGB-D、ray-depth、pose convention、室内 scene 归一化问题。⁴³
4. **uCO3D、InteriorGS、Articraft-10K**：因为这组能把系统推进到 point cloud / Gaussian / articulation 三个高级模态。⁴⁴

如果你希望这个系统后面能真正服务“自动构建目标数据集格式的数据”，我再给你一个非常关键的产品化建议：

把“数据集适配”与“数据集生成”分开。

前者负责把源数据变成中间格式；后者再根据训练任务导出特定视图，例如：

- 给检测模型：导出 COCO / BDD / KITTI-style
- 给 NeRF / 3DGS：导出 Nerfstudio / COLMAP / GS-style
- 给具身导航：导出 scene + occupancy + navmesh + object semantics
- 给地图模型：导出 image + polyline + mask + attribute table

这样你后面加新数据集，不会影响已有 exporter；你加新任务，也不用回头重写所有 adapter。

开放问题与限制

有几类边界条件，需要在第一版里明确写进系统，而不是等运行时报错：

有些数据集有访问门槛或门控下载。Matterport3D 需要签署 ToU；HM3D 下载依赖 Matterport token；MVLingNet 需要表单与密码；InteriorGS 需要接受访问条件。你的 Agent 应当在 registry 里维护 `access_mode = public | gated | agreement_required`，并在不能自动获取时输出“待人工授权清单”，而不是把流程卡死。 45

还有几项表格条目需要你在工程上额外澄清。**Maya** 缺少官方链接，现阶段更像是“自定义场景源类别”而非可唯一识别的公共 benchmark；**Wayve** 如果你指的是公开数据，当前最可核验的是 WayveScenes101；**ScanNet++** 的 **scene graph** 更适合作为导出目标，而不是官方文档中已固定好的原生标注形态。关于 **Waymo**，官方 about 页非常明确地描述了传感器、标定、位姿与 2D/3D 标注，但其原始打包形式在社区工程里经常还要再做一层读取/预转换，因此你的第一版不建议手写全量底层解析器。 46

最后，真正的复杂度往往不在“读文件”，而在这三件事：

- **坐标系**：如 Hypersim 的 asset units、ScanNet++ 的 COLMAP 与 Nerfstudio 双约定、nuScenes 的全局/ego/sensor 关系。 47
- **深度定义**：如 ASE 的 ray depth 与许多数据中常见的 z-depth 并不相同。 48
- **标签体系**：如 uCO3D 的 LVIS、Hypersim/ScanNet 的 NYU40、OpenSatMap 的 polyline attributes、BDD100K/nuScenes/KITTI 的驾驶类目体系完全不同。 49

所以，你这套 SKILL.md 最终要落成的，不是一份“下载并转换”的说明书，而是一套可验证的、面向多源三维数据的 **schema 操作系统**。只要把“格式识别—坐标归一—语义映射—profile 导出—质量校验”这五步固化下来，后续无论接入 3DGS、USD、URDF、卫星折线还是自动驾驶序列，系统都能持续扩。

1 40 [raw.githubusercontent.com](https://raw.githubusercontent.com/facebookresearch/co3d/main/co3d/dataset/data_types.py)

https://raw.githubusercontent.com/facebookresearch/co3d/main/co3d/dataset/data_types.py

2 5 48 https://facebookresearch.github.io/projectaria_tools/docs/open_datasets/aria_synthetic_environments_dataset/ase_data_format

https://facebookresearch.github.io/projectaria_tools/docs/open_datasets/aria_synthetic_environments_dataset/ase_data_format

3 8 47 [GitHub - apple/ml-hypersim: Hypersim: A Photorealistic Synthetic Dataset for Holistic Indoor Scene Understanding](https://github.com/apple/ml-hypersim) • GitHub

<https://github.com/apple/ml-hypersim>

4 [BlenderProc/examples/datasets/front_3d/README.md](https://github.com/DLR-RM/BlenderProc/blob/main/examples/datasets/front_3d/README.md) at ...

https://github.com/DLR-RM/BlenderProc/blob/main/examples/datasets/front_3d/README.md?utm_source=chatgpt.com

6 43 <https://github.com/apple/ARKitScenes>

<https://github.com/apple/ARKitScenes>

7 <https://openreview.net/forum?id=-v4OuqNs5P>

<https://openreview.net/forum?id=-v4OuqNs5P>

9 45 [GitHub - matterport/3d-dataset-tools](https://github.com/matterport/3d-dataset-tools) • GitHub

<https://github.com/matterport/3d-dataset-tools>

10 <https://github.com/facebookresearch/replica-dataset>

<https://github.com/facebookresearch/replica-dataset>

- 11 https://aihabitat.org/datasets/replica_cad/
https://aihabitat.org/datasets/replica_cad/
- 12 <https://github.com/ScanNet/ScanNet>
<https://github.com/ScanNet/ScanNet>
- 13 <https://scannetpp.mlsg.cit.tum.de/scannetpp/documentation>
<https://scannetpp.mlsg.cit.tum.de/scannetpp/documentation>
- 14 <https://huggingface.co/datasets/spatialverse/InteriorAgent>
<https://huggingface.co/datasets/spatialverse/InteriorAgent>
- 15 <https://huggingface.co/datasets/spatialverse/InteriorGS>
<https://huggingface.co/datasets/spatialverse/InteriorGS>
- 16 <https://huggingface.co/datasets/nvidia/SAGE-10k>
<https://huggingface.co/datasets/nvidia/SAGE-10k>
- 17 <https://huggingface.co/datasets/xinjue1/TabletopGen-Assets>
<https://huggingface.co/datasets/xinjue1/TabletopGen-Assets>
- 18 20 **GitHub - facebookresearch/co3d: Tooling for the Common Objects In 3D dataset.** • GitHub
<https://github.com/facebookresearch/co3d>
- 19 42 <https://github.com/lmb-freiburg/robustmvd/blob/master/rmvd/data/README.md>
<https://github.com/lmb-freiburg/robustmvd/blob/master/rmvd/data/README.md>
- 21 <https://dl3dv-10k.github.io/DL3DV-10K/>
<https://dl3dv-10k.github.io/DL3DV-10K/>
- 22 **MegaDepth: Learning Single-View Depth Prediction from Internet Photos**
<https://www.cs.cornell.edu/projects/megadepth/>
- 23 **GitHub - GAP-LAB-CUHK-SZ/MVImgNet: CVPR2023 | MVImgNet: A Large-scale Dataset of Multi-view Images** • GitHub
<https://github.com/GAP-LAB-CUHK-SZ/MVImgNet>
- 24 **MVImgNet2.0: A Larger-scale Dataset of Multi-view Images**
<https://luyues.github.io/mvimnet2/>
- 25 <https://objaverse.allenai.org/>
<https://objaverse.allenai.org/>
- 26 <https://github.com/NJU-PCALab/OpenVid-1M>
<https://github.com/NJU-PCALab/OpenVid-1M>
- 27 44 49 <https://github.com/facebookresearch/uco3d>
<https://github.com/facebookresearch/uco3d>
- 28 <https://huggingface.co/datasets/ai-habitat/ycb>
<https://huggingface.co/datasets/ai-habitat/ycb>
- 29 <https://huggingface.co/datasets/camvsl/Articraft-10K>
<https://huggingface.co/datasets/camvsl/Articraft-10K>
- 30 32 **nutonomy/nuscenes-devkit**
https://github.com/nutonomy/nuscenes-devkit?utm_source=chatgpt.com

- 31 41 https://openaccess.thecvf.com/content_CVPR_2020/papers/Yu_BDD100K_A_Diverse_Driving_Dataset_for_Heterogeneous_Multitask_Learning_CVPR_2020_paper.pdf
https://openaccess.thecvf.com/content_CVPR_2020/papers/Yu_BDD100K_A_Diverse_Driving_Dataset_for_Heterogeneous_Multitask_Learning_CVPR_2020_paper.pdf
- 33 https://www.cvlibs.net/datasets/kitti/raw_data.php
https://www.cvlibs.net/datasets/kitti/raw_data.php
- 34 <https://www.cvlibs.net/datasets/kitti-360/index.php>
<https://www.cvlibs.net/datasets/kitti-360/index.php>
- 35 46 <https://wayve.ai/science/wayvescenes101/>
<https://wayve.ai/science/wayvescenes101/>
- 36 <https://waymo.com/open/about/>
<https://waymo.com/open/about/>
- 37 <https://opensatmap.github.io/>
<https://opensatmap.github.io/>
- 38 <https://github.com/rilab314/SatelliteLaneDataset2024>
<https://github.com/rilab314/SatelliteLaneDataset2024>
- 39 <https://github.com/billbillbilly/gloBFPr>
<https://github.com/billbillbilly/gloBFPr>